TRƯỜNG ĐẠI HỌC PHÚ XUÂN
KHOA CÔNG NGHỆ THÔNG TIN

HƯỚNG DẪN THỰC HÀNH

INT.7.18 — WEB FRONTEND NÂNG CAO

Bài 3 — NODE.JS, MODULE & CÔNG CỤ FRONTEND

Mục	Nội dung
Dự án	phu-xuan-react — Node.js 20 LTS · npm 10 · Vite 5
Mã học phần	INT.7.18 — Trường Đại học Phú Xuân
Thời lượng thực hành	60 phút (3 lab × 15–20 phút + 1 lab nâng cao 15–20 phút)
Yêu cầu đầu vào	Đã hoàn thành Buổi 2: dự án phu-xuan-react chạy được bằng npm run dev tại localhost:5173
Nộp bài	Push branch feature/lab-buoi3 lên GitHub trước buổi học tiếp theo

Mục tiêu thực hành Buổi 3

Sau buổi thực hành, sinh viên sẽ:

1. Phân biệt và viết được cú pháp CommonJS (require/module.exports) và ES Modules (import/export) trong cùng một dự án.
2. Tạo và đọc file .env, dùng import.meta.env trong mã nguồn Vite, giải thích quy tắc tiền tố VITE_.
3. Cấu trúc thư mục src/ theo kiến trúc feature-based đúng chuẩn phu-xuan-react.
4. (Nâng cao) Viết máy chủ HTTP tối giản bằng Node.js, chạy song song với Vite dev server, và đọc dữ liệu JSON qua fetch() từ React.

YÊU CẦU TRƯỚC KHI BẮT ĐẦU

Checklist bắt buộc — hoàn thành từ Buổi 2

- ☐ Node.js 20 LTS đã cài — kiểm tra: node --version → phải ra v20.x.x trở lên
- ☐ Thư mục phu-xuan-react tồn tại và npm install đã chạy thành công (node_modules/ có mặt)
- ☐ npm run dev từ thư mục dự án mở được localhost:5173 trên trình duyệt
- ☐ Git đã được cấu hình (git config user.name, git config user.email)
- ☐ VS Code đã mở thư mục phu-xuan-react

Nếu bất kỳ ô nào chưa đánh dấu, hãy báo giảng viên trước khi tiếp tục — các bước sau đây phụ thuộc vào điều kiện trên.

Cấu trúc thư mục tham chiếu

Trước khi bắt đầu lab, hãy xác nhận thư mục dự án của bạn trông như sau:

Terminal — kiểm tra cấu trúc dự án

```
phu-xuan-react/
├── node_modules/          ← do npm install tạo ra, KHÔNG commit lên Git
├── public/
├── src/
│   ├── App.tsx
│   ├── main.tsx
│   └── index.css
├── .gitignore             ← phải có dòng node_modules và .env
├── package.json
├── tsconfig.json
└── vite.config.ts
```

Nếu thiếu file .gitignore, hãy tạo ngay bây giờ với nội dung tối thiểu:

file: .gitignore (đặt ở thư mục gốc phu-xuan-react)

```
node_modules
.env
.env.local
dist
.DS_Store
```

LAB 1 · CommonJS và ES Modules — Viết và so sánh trực tiếp
⏱ Thời gian: 20 phút
🎯 Mục tiêu: Tạo file tiện ích, xuất/nhập bằng cả hai cú pháp, chạy bằng node

LAB 1 — CommonJS và ES Modules

Lab này không đụng đến Vite hay React — chỉ dùng Node.js thuần trong terminal để cảm nhận sự khác biệt giữa hai cú pháp module TRƯỚC KHI đưa chúng vào dự án. Đây là cách học vững nhất: tách biệt khái niệm khỏi framework.

Bước 1: Tạo thư mục thực hành riêng ngoài dự án

Mở terminal (không phải terminal đang chạy npm run dev), điều hướng đến Desktop hoặc một thư mục làm việc tạm:

Terminal
```
# Tạo thư mục thực hành tạm — NGOÀI thư mục phu-xuan-react
mkdir ~/module-lab && cd ~/module-lab
# Khởi tạo package.json tối giản (bấm Enter tất cả)
npm init -y
```

Bước 2: Viết file tiện ích bằng CommonJS

Tạo file utils-cjs.js với nội dung sau — CHÚ Ý: đây là cú pháp CommonJS, KHÔNG dùng import/export:

file: ~/module-lab/utils-cjs.js
```
// ============================================================
// COMMONJS — cú pháp truyền thống của Node.js
// Từ khoá: require() để nhập, module.exports để xuất
// ============================================================

// Hàm 1: định dạng ngày tháng kiểu Việt Nam
function formatDate(date) {
  return date.toLocaleDateString('vi-VN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
  });
}

// Hàm 2: rút ngắn chuỗi nếu quá dài
function truncate(str, maxLength = 50) {
  if (str.length <= maxLength) return str;
  return str.slice(0, maxLength - 3) + '...';
}

// Hàm 3: tạo slug từ tiêu đề bài viết
```

```
function toSlug(title) {
  return title
    .toLowerCase()
    .replace(/\s+/g, '-')    // thay khoảng trắng bằng dấu -
    .replace(/[^a-z0-9-]/g, ''); // xoá ký tự đặc biệt
}

// XUẤT: gán vào module.exports (CommonJS)
module.exports = { formatDate, truncate, toSlug };
```

Bước 3: Viết file nhập lại theo CommonJS và chạy thử

file: ~/module-lab/dung-cjs.js

```
// NHẬP bằng require() — CommonJS
const { formatDate, truncate, toSlug } = require('./utils-cjs');

// --- Thử nghiệm 1: formatDate ---
const homNay = new Date();
console.log('Ngày hôm nay:', formatDate(homNay));
// → Ngày hôm nay: 23/08/2026

// --- Thử nghiệm 2: truncate ---
const tieuDeDai = 'Học React từ cơ bản đến nâng cao tại Đại học Phú Xuân năm 2026';
console.log('Rút ngắn:', truncate(tieuDeDai, 40));
// → Rút ngắn: Học React từ cơ bản đến nâng cao...

// --- Thử nghiệm 3: toSlug ---
const tieuDe = 'Giới thiệu React và JSX';
console.log('Slug:', toSlug(tieuDe));
// → Slug: gii-thiu-react-v-jsx
```

Terminal — chạy thử

```
node dung-cjs.js
```

Kết quả mong đợi

```
Ngày hôm nay: 23/08/2026
Rút ngắn: Học React từ cơ bản đến nâng cao...
Slug: gii-thiu-react-v-jsx
```

Bước 4: Chuyển sang ES Modules — viết lại cùng hàm, khác cú pháp

Bây giờ viết lại ĐÚNG cùng nội dung nhưng bằng cú pháp ES Modules (import/export). Đây là cú pháp dùng trong toàn bộ src/ của phu-xuan-react.

Trước tiên cần khai báo package.json là ES Modules — thêm dòng "type": "module":

Terminal — cập nhật package.json

```
# Thêm "type": "module" vào package.json
node -e "
  const p = require('fs');
  const pkg = JSON.parse(p.readFileSync('package.json', 'utf8'));
  pkg.type = 'module';
  p.writeFileSync('package.json', JSON.stringify(pkg, null, 2));
  console.log('Đã thêm type: module');
"

# Hoặc mở package.json trong VS Code và thêm thủ công:
# "type": "module"
```

file: ~/module-lab/utils-esm.js

```
//
// ============================================================
// ES MODULES — chuẩn hiện đại, dùng xuyên suốt phu-xuan-react
// Từ khoá: import để nhập, export để xuất
//
// ============================================================

// NAMED EXPORT: xuất có tên — có thể xuất nhiều trong 1 file
export function formatDate(date) {
  return date.toLocaleDateString('vi-VN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
  });
}

export function truncate(str, maxLength = 50) {
  if (str.length <= maxLength) return str;
  return str.slice(0, maxLength - 3) + '...';
}

export function toSlug(title) {
  return title
    .toLowerCase()
    .replace(/\s+/g, '-')
    .replace(/[^a-z0-9-]/g, '');
}

// DEFAULT EXPORT: xuất mặc định — chỉ 1 mỗi file, thường dùng cho
class/object tổng hợp
const StringUtils = { formatDate, truncate, toSlug };
export default StringUtils;
```

file: ~/module-lab/dung-esm.js

```
// NHẬP named exports — phải đúng tên, phải có { }
import { formatDate, truncate, toSlug } from './utils-esm.js';
```

```
// NHẬP default export — đặt tên tuỳ ý, KHÔNG có { }
import Utils from './utils-esm.js';

// --- Dùng named imports ---
console.log('Named: ngày hôm nay:', formatDate(new Date()));

// --- Dùng default import ---
console.log('Default: slug:', Utils.toSlug('Xin chào phu-xuan-react'));

// --- So sánh: cùng kết quả, khác cách nhập ---
const tieu = 'React nâng cao 2026';
console.log('Named  truncate:', truncate(tieu, 15));
console.log('Default truncate:', Utils.truncate(tieu, 15));
```

Terminal — chạy thử ESM

```
node dung-esm.js
```

Kết quả mong đợi

```
Named: ngày hôm nay: 23/08/2026
Default: slug: xin-cho-phu-xuan-react
Named  truncate: React nâng cao...
Default truncate: React nâng cao...
```

Bước 5: Chuyển utils-esm.js vào dự án phu-xuan-react

Bước cuối cùng: copy file tiện ích vừa viết vào đúng vị trí trong phu-xuan-react. Đây là lần đầu sinh viên thêm một module tự viết vào dự án thật.

Terminal — copy file vào dự án

```
# Đứng trong ~/module-lab
cp utils-esm.js ~/path/to/phu-xuan-react/src/utils/string-utils.ts
# (thay ~/path/to/phu-xuan-react bằng đường dẫn thật trên máy bạn)

# Nếu thư mục src/utils chưa có:
mkdir -p ~/path/to/phu-xuan-react/src/utils
```

Mở src/App.tsx và thêm dòng import để kiểm tra Vite nhận ra module mới:

file: src/App.tsx — thêm ở đầu file, sau dòng import React

```
import { formatDate, toSlug } from './utils/string-utils';

// Thêm tạm vào trong hàm App() để kiểm tra:
console.log('Hôm nay:', formatDate(new Date()));
console.log('Slug:', toSlug('Trang chủ phu-xuan-react'));
```

Lưu file, mở console của trình duyệt (F12 → Console) — nếu thấy hai dòng log, lab 1 hoàn thành.

✓ CHECKPOINT: Lab 1 hoàn thành — xác nhận trước khi sang Lab tiếp theo

☐ Terminal 1 (CommonJS): chạy node dung-cjs.js → thấy 3 dòng kết quả đúng

☐ Terminal 2 (ES Modules): chạy node dung-esm.js → thấy 4 dòng kết quả đúng

☐ VS Code: src/utils/string-utils.ts tồn tại trong phu-xuan-react

☐ Trình duyệt Console: thấy 2 dòng log từ App.tsx khi npm run dev đang chạy

Lỗi / Triệu chứng	Nguyên nhân có thể	Cách khắc phục
SyntaxError: Cannot use import statement in a module (với file .js)	package.json chưa có "type": "module"	Thêm "type": "module" vào package.json HOẶC đổi phần mở rộng file thành .mjs
Error: require is not defined in ES module scope	File dùng require() nhưng package.json đã có "type": "module"	Đổi require() sang import, hoặc dùng createRequire() từ module 'module'
Cannot find module './utils-esm.js'	ES Modules yêu cầu phần mở rộng .js đầy đủ khi chạy bằng node	Thêm .js vào đường dẫn: import ... from './utils-esm.js'
Module './utils/string-utils' not found (trong Vite)	Đường dẫn tương đối sai hoặc thư mục utils chưa tồn tại	Kiểm tra lại đường dẫn: ls src/utils/ trong terminal

LAB 2 · Biến môi trường — .env và import.meta.env trong Vite
⏱ Thời gian: 15 phút
🎯 Mục tiêu: Tạo và đọc biến môi trường, hiểu quy tắc VITE_ và bảo mật

LAB 2 — Biến môi trường và cấu hình dự án

Lab này thực hiện hoàn toàn trong thư mục phu-xuan-react. Đảm bảo npm run dev đang chạy ở một terminal riêng — bạn sẽ thấy kết quả trực tiếp trên trình duyệt sau mỗi thay đổi nhờ Hot Reload.

Bước 1: Tạo file .env ở thư mục gốc dự án

File .env phải đặt ở cùng cấp với package.json — KHÔNG đặt trong src/:

file: phu-xuan-react/.env (tạo mới, đặt ở thư mục gốc)

```
# Địa chỉ máy chủ API — dùng trong môi trường phát triển (development)
VITE_API_BASE_URL=http://localhost:4000

# Tên ứng dụng — hiển thị trong tiêu đề trang
VITE_APP_TITLE=phu-xuan-react

# Phiên bản — cũng hiển thị được trong UI
VITE_APP_VERSION=1.0.0

# ⚠ Biến KHÔNG có tiền tố VITE_ → Vite sẽ KHÔNG đưa vào code trình duyệt
DATABASE_URL=postgresql://localhost/phu_xuan_db
SECRET_KEY=khoa-bi-mat-tuyet-doi-khong-duoc-commit
```

Quan trọng: .env đã có trong .gitignore chưa?

Mở file .gitignore ở thư mục gốc và kiểm tra dòng .env có mặt không.

Nếu chưa có, thêm ngay — một khi .env chứa khoá thật đã được push lên GitHub, bạn không thể xoá hoàn toàn dù đã xoá commit (Git lưu lịch sử, GitHub có thể đã cache).

Kiểm tra: git status → không nên thấy .env trong danh sách tracked files.

Bước 2: Đọc biến môi trường trong mã nguồn React

Mở src/App.tsx và thêm đoạn code đọc biến môi trường. LƯU Ý: phải dừng và khởi động lại npm run dev sau khi tạo/sửa .env — Vite chỉ đọc .env một lần khi khởi động.

file: src/App.tsx — thêm vào sau các dòng import hiện có

```
// Đọc biến môi trường — Vite thay thế các dòng này bằng giá trị thật lúc build
const apiUrl  = import.meta.env.VITE_API_BASE_URL;
const appTitle = import.meta.env.VITE_APP_TITLE;
const version  = import.meta.env.VITE_APP_VERSION;
```

```
// Biến môi trường đặc biệt của Vite (luôn có sẵn, không cần khai báo trong .env):
const isDev  = import.meta.env.DEV;   // true khi chạy npm run dev
const isProd = import.meta.env.PROD;  // true khi chạy npm run build
const mode   = import.meta.env.MODE;  // 'development' hoặc 'production'
```

file: src/App.tsx — thay nội dung return() trong hàm App bằng:

```
return (
  <div style={{ padding: '2rem', fontFamily: 'sans-serif' }}>
    <h1>{appTitle} — v{version}</h1>
    <hr />
    <h2>Thông tin môi trường</h2>
    <table border={1} cellPadding={8}>
      <tbody>
        <tr><td>API URL</td><td><code>{apiUrl}</code></td></tr>
        <tr><td>Chế độ</td><td><code>{mode}</code></td></tr>
        <tr><td>Đang phát triển?</td><td>{isDev ? '✓ Có' : '✗ Không'}</td></tr>
        <tr><td>Đang production?</td><td>{isProd ? '✓ Có' : '✗ Không'}</td></tr>
      </tbody>
    </table>
    <hr />
    <h2>Kiểm tra bảo mật</h2>
    <p>
      DATABASE_URL (không có VITE_):{' '}
      <code>{String(import.meta.env.DATABASE_URL)}</code>
    </p>
    <p style={{ color: 'green' }}>
      Nếu thấy 'undefined' ở trên → Vite đã bảo vệ biến bí mật đúng cách ✓
    </p>
  </div>
);
```

Bước 3: Khởi động lại Vite và quan sát

Terminal (terminal đang chạy npm run dev) — dừng và chạy lại

```
# Bấm Ctrl+C để dừng
^C

# Chạy lại để Vite đọc .env mới
npm run dev
```

Mở localhost:5173 trên trình duyệt. Bạn sẽ thấy:

```
# Kết quả mong đợi
phu-xuan-react — v1.0.0

Thông tin môi trường:
  API URL     | http://localhost:4000
  Chế độ      | development
```

```
  Đang phát triển? | ✓ Có
  Đang production? | ✗ Không

Kiểm tra bảo mật:
  DATABASE_URL (không có VITE_): undefined  ← Vite đã bảo vệ đúng cách ✓
```

Bước 4: Tạo file .env.local cho cấu hình riêng từng máy

Thực tế trong nhóm: mỗi thành viên có thể có cấu hình địa chỉ API khác nhau (máy chạy cổng khác, máy chạy trên localhost khác...). .env.local cho phép ghi đè .env mà không bị commit lên Git.

file: phu-xuan-react/.env.local (tạo mới — đã có trong .gitignore)

```
# Ghi đè .env cho máy này — ưu tiên cao hơn .env
VITE_API_BASE_URL=http://localhost:3001
VITE_APP_TITLE=phu-xuan-react (máy của tôi)
```

Không cần khởi động lại — thêm .env.local và Ctrl+C + npm run dev để thấy giá trị mới. Quan sát: API URL đã thay đổi thành :3001, title đã thay đổi.

Thứ tự ưu tiên biến môi trường Vite

.env.local (ưu tiên cao nhất — dành cho từng máy, không commit)

.env.[mode].local (ví dụ .env.development.local)

.env.[mode] (ví dụ .env.development)

.env (ưu tiên thấp nhất — commit lên Git, dùng giá trị mặc định cho cả nhóm)

✓ CHECKPOINT: Lab 2 hoàn thành — xác nhận trước khi sang Lab tiếp theo

☐ Trang localhost:5173 hiển thị tiêu đề và bảng thông tin môi trường

☐ DATABASE_URL hiển thị 'undefined' → Vite đã bảo vệ biến không có tiền tố VITE_

☐ .env và .env.local đều có trong .gitignore (kiểm tra: git status không liệt kê 2 file này)

☐ Sau khi tạo .env.local, npm run dev hiển thị giá trị từ .env.local thay vì .env

Lỗi / Triệu chứng	Nguyên nhân có thể	Cách khắc phục
Biến hiển thị 'undefined' dù đã khai báo trong .env	Chưa khởi động lại npm run dev sau khi sửa .env	Ctrl+C → npm run dev

Lỗi / Triệu chứng	Nguyên nhân có thể	Cách khắc phục
Biến DATABASE_URL (không có VITE_) hiển thị được giá trị thật	Tên biến bắt đầu sai — có thể đang đọc nhầm biến khác	Kiểm tra tên biến trong .env — phải KHÔNG có VITE_ để bị ẩn
.env bị liệt kê trong git status	.gitignore chưa có dòng .env	Thêm .env vào .gitignore; nếu đã commit: git rm --cached .env
import.meta.env không tồn tại (lỗi TypeScript)	Vite có type declaration chưa được nhận	Kiểm tra tsconfig.json có include vite/client; thêm nếu thiếu

LAB 3 · Cấu trúc thư mục — Tổ chức module theo kiến trúc Feature-based
⏱ Thời gian: 15 phút
🎯 Mục tiêu: Xây dựng kiến trúc thư mục src/ chuẩn, thực hành re-export barrel file

LAB 3 — Kiến trúc thư mục src/ và Barrel Exports

Một trong những thách thức thực tế khi dự án phát triển là quản lý import path — khi có hàng chục component, các dòng import trở nên dài và khó bảo trì. Lab này xây dựng kiến trúc thư mục đúng chuẩn cho phu-xuan-react và giới thiệu kỹ thuật barrel export để rút ngắn đường dẫn import.

Bước 1: Tạo cấu trúc thư mục src/ theo kiến trúc feature-based

Mở terminal trong thư mục phu-xuan-react và chạy các lệnh sau:

Terminal — tạo cấu trúc thư mục

```
cd src

# Tầng components: các thành phần UI tái sử dụng
mkdir -p components/ui
mkdir -p components/layout

# Tầng features: mỗi tính năng nghiệp vụ trong một thư mục riêng
mkdir -p features/posts
mkdir -p features/users

# Tầng utils: hàm tiện ích thuần (không phụ thuộc React)
mkdir -p utils

# Tầng services: logic gọi API
mkdir -p services

# Tầng hooks: React hooks tùy biến
mkdir -p hooks

# Tầng types: khai báo TypeScript type/interface dùng chung
mkdir -p types

# Tầng constants: hằng số toàn ứng dụng
mkdir -p constants
```

Kết quả mong đợi sau khi tạo — chạy lệnh kiểm tra:

Terminal — kiểm tra cấu trúc đã tạo

```
# Đứng trong thư mục phu-xuan-react
find src -type d | sort
```

```
# Kết quả mong đợi
src
src/components
src/components/layout
src/components/ui
src/constants
src/features
src/features/posts
src/features/users
src/hooks
src/services
src/types
src/utils
```

Bước 2: Tạo file types dùng chung — khai báo TypeScript interface

file: src/types/index.ts

```
// Kiểu dữ liệu dùng chung toàn ứng dụng phu-xuan-react

// Bài viết
export interface Post {
  id: number;
  title: string;
  excerpt: string;       // tóm tắt ngắn
  content: string;       // nội dung đầy đủ
  authorId: number;
  publishedAt: string;    // ISO 8601 string
  tags: string[];
}

// Người dùng
export interface User {
  id: number;
  name: string;
  email: string;
  avatarUrl?: string;     // dấu ? = có thể không có
}

// Trạng thái tải dữ liệu — dùng cho mọi API call
export type LoadingState = 'idle' | 'loading' | 'success' | 'error';

// Phản hồi API tổng quát (generic)
export interface ApiResponse<T> {
  data: T;
  message: string;
  success: boolean;
}
```

Bước 3: Tạo constants — hằng số tập trung

file: src/constants/index.ts

```
// Tất cả hằng số tập trung ở đây — không rải rác trong component

// Địa chỉ API — lấy từ biến môi trường (đã học Lab 2)
export const API_BASE_URL = import.meta.env.VITE_API_BASE_URL ??
'http://localhost:4000';

// Cấu hình phân trang
export const POSTS_PER_PAGE = 10;

// Thời gian cache (milliseconds)
export const CACHE_TTL = 5 * 60 * 1000; // 5 phút

// Độ dài tối đa
export const MAX_TITLE_LENGTH = 120;
export const MAX_EXCERPT_LENGTH = 200;

// Nhãn hiển thị
export const APP_NAME = import.meta.env.VITE_APP_TITLE ??
'phu-xuan-react';
```

Bước 4: Tạo utils — chuyển file từ Lab 1 vào đúng vị trí

file: src/utils/string-utils.ts

```
import { MAX_TITLE_LENGTH, MAX_EXCERPT_LENGTH } from
'../constants';

// Định dạng ngày tháng kiểu Việt Nam
export function formatDate(date: Date | string): string {
  const d = typeof date === 'string' ? new Date(date) : date;
  return d.toLocaleDateString('vi-VN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
  });
}

// Rút ngắn chuỗi — dùng hằng số từ constants
export function truncateTitle(str: string): string {
  if (str.length <= MAX_TITLE_LENGTH) return str;
  return str.slice(0, MAX_TITLE_LENGTH - 3) + '...';
}

export function truncateExcerpt(str: string): string {
  if (str.length <= MAX_EXCERPT_LENGTH) return str;
```

```
  return str.slice(0, MAX_EXCERPT_LENGTH - 3) + '...';
}

// Tạo slug URL từ tiêu đề tiếng Việt (đơn giản hoá — không dùng thư viện ngoài)
export function toSlug(title: string): string {
  return title
    .toLowerCase()
    .replace(/\s+/g, '-')
    .replace(/[^a-z0-9-]/g, '');
}
```

Bước 5: Barrel export — kỹ thuật rút ngắn đường dẫn import

Khi có nhiều file trong một thư mục, thay vì import từng file riêng lẻ, ta tạo một file index.ts tổng hợp tất cả — gọi là barrel export. Kết quả: thay vì viết đường dẫn dài, chỉ cần import từ tên thư mục.

TRƯỚC khi có barrel export — phải viết đầy đủ từng đường dẫn

```
import { formatDate } from '../../utils/string-utils';
import { Post } from '../../types/index';
import { API_BASE_URL } from '../../constants/index';
```

file: src/utils/index.ts (barrel export)

```
// Re-export tất cả từ các file trong utils/
export * from './string-utils';

// Nếu sau này thêm date-utils.ts, format-utils.ts... chỉ cần thêm dòng ở đây
// export * from './date-utils';
```

file: src/types/index.ts — đã có sẵn từ Bước 2, không cần sửa

```
// (Không có thay đổi — file đã tạo đầy đủ ở Bước 2)
```

SAU khi có barrel export — import gọn hơn nhiều

```
// Lấy từ thư mục, không cần chỉ tên file cụ thể
import { formatDate, toSlug } from '../utils';
import { Post, User } from '../types';
import { API_BASE_URL, APP_NAME } from '../constants';
```

Bước 6: Kiểm tra: cập nhật App.tsx dùng kiến trúc mới

file: src/App.tsx — viết lại để dùng kiến trúc mới

```
import { formatDate, toSlug } from './utils';
import { APP_NAME } from './constants';
import type { Post } from './types';

// Dữ liệu mẫu tạm — sẽ thay bằng API thật từ Buổi 17
const samplePost: Post = {
```

```
  id: 1,
  title: 'Giới thiệu React và hệ sinh thái hiện đại',
  excerpt: 'Bài viết đầu tiên của phu-xuan-react giới thiệu các khái niệm cốt
lõi.',
  content: 'Nội dung đầy đủ sẽ được tải từ API...',
  authorId: 1,
  publishedAt: new Date().toISOString(),
  tags: ['react', 'javascript', 'frontend'],
};

export default function App() {
  return (
    <div style={{ padding: '2rem', fontFamily: 'sans-serif', maxWidth: '800px'
}}>
      <h1>{APP_NAME}</h1>
      <hr />
      <article>
        <h2>{samplePost.title}</h2>
        <p style={{ color: 'gray' }}>
          Đăng ngày: {formatDate(samplePost.publishedAt)}
        </p>
        <p>{samplePost.excerpt}</p>
        <p>
          Slug URL: <code>/{toSlug(samplePost.title)}</code>
        </p>
        <div>
          Tags: {samplePost.tags.map(tag => (
            <span key={tag} style={{ marginRight: 8, background: '#eee',
padding: '2px 8px', borderRadius: 4 }}>
              {tag}
            </span>
          ))}
        </div>
      </article>
    </div>
  );
}
```

Kết quả mong đợi

Trang web hiển thị:
phu-xuan-react ← từ APP_NAME
Giới thiệu React và hệ sinh thái hiện đại
Đăng ngày: [ngày hôm nay dạng dd/mm/yyyy] ← từ formatDate()
[đoạn excerpt]
Slug URL: /gii-thiu-react-v-h-sinh-thi-hin-i ← từ toSlug()
Tags: [react] [javascript] [frontend]

✓ CHECKPOINT: Lab 3 hoàn thành — xác nhận trước khi sang Lab tiếp theo

☐ Cấu trúc thư mục src/ đúng như sơ đồ (kiểm tra bằng find src -type d | sort)

☐ src/types/index.ts định nghĩa đủ 4 kiểu: Post, User, LoadingState, ApiResponse

☐ src/utils/index.ts có barrel export → import { formatDate } from './utils' hoạt động

☐ localhost:5173 hiển thị bài viết mẫu với ngày định dạng tiếng Việt và slug URL đúng

Lỗi / Triệu chứng	Nguyên nhân có thể	Cách khắc phục
TypeScript lỗi 'Cannot find module' dù đường dẫn nhìn đúng	barrel export chưa re-export hàm cần dùng	Kiểm tra src/utils/index.ts có dòng export * from './string-utils'
Lỗi 'Module ... has no exported member ...'	Tên export không khớp (ví dụ đặt tên khác nhau ở file gốc và barrel)	Xem lại tên export chính xác trong file gốc
import.meta.env.VITE_... undefined trong constants/index.ts	Chưa khởi động lại npm run dev sau khi cập nhật .env	Ctrl+C và npm run dev lại
toSlug() cho kết quả không có dấu tiếng Việt	Hàm regex chưa xử lý ký tự Unicode — bình thường ở bước này	Sẽ bổ sung thư viện slug chuyên dụng ở giai đoạn sau; bây giờ chấp nhận kết quả đơn giản

LAB 4 · Nâng cao — Máy chủ HTTP Node.js & Gọi dữ liệu từ React
⏱ Thời gian: 20 phút 🎯 Mục tiêu: Viết API server 15 dòng, fetch JSON từ React, hiểu CORS và Request–Response

LAB 4 (Nâng cao) — Máy chủ HTTP Node.js chạy song song Vite

Dành cho ai?

Lab này dành cho sinh viên đã hoàn thành Lab 1–3 và còn thời gian. Đây là bản rút gọn của máy chủ API giả lập mà cả lớp sẽ dùng từ Buổi 17 — làm quen trước giúp bạn hiểu cơ chế hoàn toàn, không phải dùng như hộp đen.

Nếu chưa kịp hoàn thành Lab 1–3, hãy hoàn thành chúng trước — Lab 4 là BONUS không bắt buộc.

Bước 1: Tạo file máy chủ HTTP trong dự án

Máy chủ này sẽ chạy SONG SONG với Vite dev server — hai tiến trình trên hai cổng khác nhau: Vite ở 5173, máy chủ API ở 4000.

file: phu-xuan-react/server/api.js (tạo thư mục server/ ở thư mục gốc)

```
// Máy chủ HTTP tối giản — mô phỏng cấu trúc máy chủ API thật
// DÙNG CommonJS (require) vì đây là file Node.js thuần, nằm ngoài src/
const http = require('http');

// Dữ liệu mẫu — sau này thay bằng database thật
const posts = [
  { id: 1, title: 'Giới thiệu React', excerpt: 'Bài viết đầu tiên về React.',
    authorId: 1, publishedAt: '2026-08-23T08:00:00Z', tags: ['react'] },
  { id: 2, title: 'ES Modules trong thực tế', excerpt: 'CommonJS và ESM khác
nhau thế nào?',
    authorId: 1, publishedAt: '2026-08-20T09:00:00Z', tags: ['javascript',
'module'] },
  { id: 3, title: 'Vite là gì?', excerpt: 'Tìm hiểu công cụ dựng nhanh nhất hiện
nay.',
    authorId: 2, publishedAt: '2026-08-18T10:00:00Z', tags: ['vite', 'tooling'] },
];

// Headers dùng chung cho mọi phản hồi JSON
function jsonHeaders() {
  return {
    'Content-Type': 'application/json; charset=utf-8',
    // CORS: cho phép Vite dev server (localhost:5173) gọi sang máy chủ này
    'Access-Control-Allow-Origin': 'http://localhost:5173',
    'Access-Control-Allow-Methods': 'GET, POST, OPTIONS',
    'Access-Control-Allow-Headers': 'Content-Type',
```

```
  };
}

const server = http.createServer((req, res) => {
  const url = req.url ?? '/';
  const method = req.method ?? 'GET';

  // Xử lý preflight CORS (trình duyệt gửi OPTIONS trước khi gửi
GET/POST thật)
  if (method === 'OPTIONS') {
    res.writeHead(204, jsonHeaders());
    res.end();
    return;
  }

  console.log(`[API] ${method} ${url}`); // log mọi request

  // GET /api/posts → trả về danh sách tất cả bài viết
  if (url === '/api/posts' && method === 'GET') {
    res.writeHead(200, jsonHeaders());
    res.end(JSON.stringify({ success: true, data: posts }));
    return;
  }

  // GET /api/posts/:id → trả về 1 bài viết theo id
  const matchPost = url.match(/^\/api\/posts\/(\d+)$/);
  if (matchPost && method === 'GET') {
    const id = parseInt(matchPost[1], 10);
    const post = posts.find(p => p.id === id);
    if (post) {
      res.writeHead(200, jsonHeaders());
      res.end(JSON.stringify({ success: true, data: post }));
    } else {
      res.writeHead(404, jsonHeaders());
      res.end(JSON.stringify({ success: false, message: 'Không tìm thấy bài
viết' }));
    }
    return;
  }

  // Tất cả route khác → 404
  res.writeHead(404, jsonHeaders());
  res.end(JSON.stringify({ success: false, message: `Route '${url}' không tồn
tại` }));
});

const PORT = process.env.PORT || 4000;
server.listen(PORT, () => {
```

```
  console.log(`✓ API server đang chạy tại http://localhost:${PORT}`);
  console.log(`  GET http://localhost:${PORT}/api/posts`);
  console.log(`  GET http://localhost:${PORT}/api/posts/1`);
});
```

Bước 2: Thêm script npm để khởi động máy chủ

Mở package.json và thêm script mới để tiện khởi động:

file: package.json — thêm vào mục "scripts"

```
{
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview",
    "api": "node server/api.js"
  }
}
```

Terminal 2 (mở terminal MỚI, giữ Terminal 1 đang chạy npm run dev)

```
npm run api
# Kết quả mong đợi:
# ✓ API server đang chạy tại http://localhost:4000
#   GET http://localhost:4000/api/posts
#   GET http://localhost:4000/api/posts/1
```

Mở trình duyệt, kiểm tra trực tiếp:

Thử trong trình duyệt hoặc curl

```
# Mở URL này trong trình duyệt:
http://localhost:4000/api/posts

# Hoặc dùng curl trong Terminal 3:
curl http://localhost:4000/api/posts
curl http://localhost:4000/api/posts/2
curl http://localhost:4000/api/posts/99   # → 404
```

Bước 3: Gọi API từ React — fetch() và xử lý trạng thái

Bây giờ viết React component gọi API server vừa tạo. Đây là phiên bản đơn giản nhất của data fetching — sau này sẽ dùng TanStack Query cho chuẩn hơn (Buổi 17).

file: src/features/posts/PostList.tsx

```
import { useState, useEffect } from 'react';
import type { Post, LoadingState, ApiResponse } from '../../types';
import { formatDate, truncateExcerpt } from '../../utils';
import { API_BASE_URL } from '../../constants';
```

```
export default function PostList() {
  const [posts, setPosts] = useState<Post[]>([]);
  const [status, setStatus] = useState<LoadingState>('idle');
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    // Bắt đầu tải → đặt trạng thái 'loading'
    setStatus('loading');
    setError(null);

    fetch(`${API_BASE_URL}/api/posts`)
      .then(res => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json() as Promise<ApiResponse<Post[]>>;
      })
      .then(json => {
        if (json.success) {
          setPosts(json.data);
          setStatus('success');
        } else {
          throw new Error(json.message);
        }
      })
      .catch(err => {
        setError(String(err.message));
        setStatus('error');
      });
  }, []); // [] = chỉ chạy một lần khi component mount

  // --- Xử lý từng trạng thái ---
  if (status === 'idle' || status === 'loading') {
    return <p>⏳ Đang tải danh sách bài viết...</p>;
  }

  if (status === 'error') {
    return (
      <div style={{ color: 'red' }}>
        <p>⚠ Không thể tải dữ liệu: {error}</p>
        <p>Hãy kiểm tra máy chủ API đang chạy tại {API_BASE_URL}</p>
      </div>
    );
  }

  return (
    <section>
      <h2>Danh sách bài viết ({posts.length} bài)</h2>
      <ul style={{ listStyle: 'none', padding: 0 }}>
```

```
      {posts.map(post => (
        <li key={post.id} style={{ border: '1px solid #ddd', borderRadius: 8,
          padding: '1rem', marginBottom: '1rem' }}>
          <h3>{post.title}</h3>
          <p style={{ color: 'gray', fontSize: '0.9em' }}>
            Đăng ngày: {formatDate(post.publishedAt)}
          </p>
          <p>{truncateExcerpt(post.excerpt)}</p>
          <div>
            {post.tags.map(tag => (
              <span key={tag} style={{ marginRight: 6, background: '#f0f0f0',
                padding: '2px 8px', borderRadius: 4, fontSize: '0.85em' }}>
                #{tag}
              </span>
            ))}
          </div>
        </li>
      ))}
    </ul>
  </section>
 );
}
```

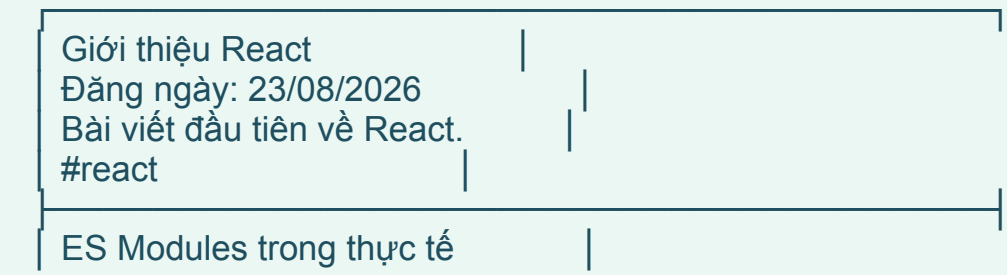
file: src/App.tsx — import và dùng PostList

```
import { APP_NAME } from './constants';
import PostList from './features/posts/PostList';

export default function App() {
 return (
  <div style={{ padding: '2rem', fontFamily: 'sans-serif', maxWidth: '800px',
margin: '0 auto' }}>
    <h1>{APP_NAME}</h1>
    <PostList />
  </div>
 );
}
```

Kết quả mong đợi

Trang web hiển thị:
phu-xuan-react
Danh sách bài viết (3 bài)





Bước 4: Quan sát CORS trong thực tế và phân tích log

CORS (Cross-Origin Resource Sharing) là cơ chế trình duyệt chặn request từ một origin (localhost:5173) sang origin khác (localhost:4000). Hãy quan sát để hiểu cách header CORS hoạt động.

Mở DevTools (F12) → tab Network → tìm request /api/posts

Request Headers (trình duyệt gửi đi):
  Origin: http://localhost:5173
  Referer: http://localhost:5173/

Response Headers (máy chủ trả về):
  Access-Control-Allow-Origin: http://localhost:5173 ← máy chủ cho phép origin này
  Content-Type: application/json; charset=utf-8
  Status: 200 OK

Bây giờ thử nghiệm xóa header CORS để thấy lỗi thật — mở server/api.js, comment dòng Access-Control-Allow-Origin, lưu và khởi động lại npm run api:

Terminal 2 (API server) — khởi động lại sau khi sửa

```
# Ctrl+C để dừng
npm run api
```

Kết quả mong đợi khi XÓA header CORS

Console trình duyệt sẽ hiển thị lỗi đỏ:

Access to fetch at 'http://localhost:4000/api/posts' from origin 'http://localhost:5173' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.

→ Đây là lỗi CORS thật — trình duyệt chặn response, code React nhận được lỗi fetch.

Sau khi quan sát xong, hãy khôi phục lại dòng header đã comment và restart API server.

✓ CHECKPOINT: Lab 4 hoàn thành — xác nhận trước khi sang Lab tiếp theo

- ☐ Terminal 2: npm run api chạy và in ra 3 dòng thông báo cổng 4000
- ☐ localhost:4000/api/posts trả về JSON hợp lệ khi mở trên trình duyệt
- ☐ localhost:5173 hiển thị danh sách 3 bài viết từ API
- ☐ DevTools Network tab: thấy request /api/posts với status 200 và header CORS

☐ (Nếu làm Bước 4) Sau khi xóa header CORS: thấy lỗi đỏ trong Console → hiểu tại sao cần CORS

Lỗi / Triệu chứng	Nguyên nhân có thể	Cách khắc phục
TypeError: Failed to fetch	npm run api chưa chạy, hoặc API đang chạy ở cổng khác	Kiểm tra Terminal 2 có chạy không; kiểm tra VITE_API_BASE_URL trong .env khớp cổng
CORS error trong Console	Header Access-Control-Allow-Origin chưa được set đúng origin	Kiểm tra dòng 'Access-Control-Allow-Origin' trong server/api.js trùng với localhost:5173
Trang hiển thị mãi 'Đang tải...'	fetch() đang pending — API server chưa phản hồi	Mở DevTools → Network, xem request /api/posts đang ở trạng thái nào
JSON.stringify lỗi circular structure	Data chứa tham chiếu vòng (circular reference)	Dùng JSON.stringify(data, null, 2) và kiểm tra cấu trúc data trước khi gửi
port 4000 already in use	Một tiến trình khác đang dùng cổng 4000	Đổi PORT trong .env hoặc dừng tiến trình cũ: lsof -ti:4000 | xargs kill

BẢNG TEST CASE TỰ KIỂM TRA

Test case	Thao tác	Kết quả mong đợi	Kết quả thực tế
[Lab 1] CommonJS chạy được	Terminal: node dung-cjs.js	3 dòng output đúng định dạng	
[Lab 1] ES Modules chạy được	Terminal: node dung-esm.js	4 dòng output, named và default import đều đúng	
[Lab 1] Import vào Vite	Thêm import trong App.tsx, kiểm tra console trình duyệt	Không lỗi TypeScript, console log hiển thị giá trị	
[Lab 2] Biến VITE_ hiển thị	localhost:5173 sau npm run dev	Bảng thông tin hiển thị URL và mode đúng	
[Lab 2] Biến không có VITE_ bị ẩn	Kiểm tra DATABASE_URL trong trình duyệt	'undefined' — Vite đã bảo vệ	
[Lab 2] .env không trong git	git status	.env không xuất hiện trong danh sách	
[Lab 3] Cấu trúc thư mục	find src -type d | sort	8 thư mục theo đúng kiến trúc	
[Lab 3] Barrel export hoạt động	import { formatDate } from './utils' trong App.tsx	Không lỗi, formatDate chạy đúng	
[Lab 3] Post mẫu hiển thị	localhost:5173	Tiêu đề, ngày định dạng, slug, tags hiển thị đúng	
[Lab 4] API server chạy	npm run api	Log 3 dòng, cổng 4000 lắng nghe	
[Lab 4] /api/posts trả JSON	localhost:4000/api/posts trong trình duyệt	JSON có success:true và mảng 3 bài	
[Lab 4] React hiển thị từ API	localhost:5173 sau khi api và dev cùng chạy	3 bài viết có tiêu đề, ngày, tags	